



UNIVERSIDAD
CATÓLICA
BOLIVIANA
ERIGIDA CANÓNICAMENTE
POR LA SANTA SEDE DESDE 2023

Maestría en Ciencia de Datos e Inteligencia Artificial Aplicada
Módulo: Visión Computacional

Detección y clasificación de defectos en superficies metálicas con aprendizaje profundo: del entrenamiento en GPU a una demo web en tiempo real

Integrantes

Simon Alex Rodriguez Saavedra	<i>Integrador</i>
Jennifer Suarez Gutierrez	<i>Entrenamiento de la red CNN baseline</i>
Daniel Ribera Añez	<i>Contribuidor</i>
Daniela Alejandra Caro Arrazola	<i>Contribuidora</i>

Agosto 2026

Repositorio del proyecto (código, datos y demo):
<https://github.com/SimonLexRS/vision-computacional>

Resumen

La inspección visual de defectos superficiales en piezas metálicas (grietas, perforaciones, óxido y rayones) es una tarea crítica del control de calidad industrial, hoy realizada en gran parte de forma manual. Este proyecto construye un sistema completo de visión computacional para automatizarla: dada una imagen —o el video en vivo de una cámara— de una superficie metálica, el sistema decide si la pieza es normal o presenta defectos, y localiza cada defecto con su caja y tipo. La solución combina dos modelos en cascada: un clasificador de dominio MobileNetV3-Small (5 clases) que actúa como *gate* y un detector YOLOv8n (4 clases de defecto). El entrenamiento se realizó en GPU (NVIDIA RTX 5060 Ti) sobre un dataset combinado de tres fuentes: 15 000 imágenes sintéticas (Kaggle, CC BY 4.0), 480 imágenes reales de SteelDefectX y 1 770 imágenes reales de NEU-DET con cajas anotadas. Los clasificadores por transfer learning alcanzan $F1 \text{ macro} = 1.0000$ en el split de test; el detector logra $mAP50 = 0.739$ sobre un test que incluye imágenes reales anotadas. Ambos modelos se exportan a ONNX y corren 100 % en el navegador (ONNX Runtime Web, WASM) a 20–30 FPS, sin servidor.

1. Introducción y motivación

Problema concreto. Dada una imagen de una superficie metálica, identificar automáticamente si presenta defectos —grieta, perforación, óxido o rayón— y *dónde* está localizado cada uno, para apoyar la decisión de aceptar o reprobar la pieza.

Caso real de referencia. Una línea de producción de acero laminado (o de piezas metálicas en general), donde hoy un operario inspecciona visualmente cada pieza bajo lámpara. Es el mismo escenario de los benchmarks industriales NEU-DET [1] y SteelDefectX [2]: inspección de defectos superficiales en tiras de acero.

Por qué importa. La inspección manual es lenta, subjetiva y se degrada con la fatiga; un detector automático y en tiempo real permite al operario (o al sistema) detener la línea, reprobar la pieza o marcarla para revisión. Que el sistema corra íntegramente en el navegador, sin enviar imágenes a un servidor, lo hace desplegable en planta sin infraestructura adicional y sin exponer datos del proceso.

Aporte del proyecto. Un pipeline completo y reproducible —descarga de datos, preparación de datasets, entrenamiento en GPU, evaluación, exportación a ONNX y verificación de paridad numérica— que culmina en una demo web en tiempo real con cámara, zoom de inspección y filtros anti-ruido calibrados.

2. Datos

Se combinan tres fuentes complementarias (Tabla 1), mapeadas a una taxonomía común de 5 clases para clasificación (**crack**, **hole**, **normal**, **rust**, **scratch**) y 4 clases de defecto para detección (sin **normal**).

Cuadro 1: Fuentes de datos del proyecto.

Fuente	Imágenes	Tipo	Anotación / licencia
Synthetic Industrial Metal Surface Defects (Abbas, Kaggle) [3]	15 000	Sintética	Metadata de posición/tamaño; cajas derivadas con OpenCV. CC BY 4.0.
SteelDefectX (Zhao et al., 2024) [2]	480	Real	Clase por imagen; cajas estimadas con OpenCV. Uso académico.
NEU-DET (Song & Yan, 2013) [1]	1 770	Real	Cajas reales anotadas (Pascal VOC XML), 6 clases mapeadas a 4. Uso académico.

Tamaño y división. El dataset de clasificación usa las 15 000 imágenes sintéticas balanceadas (3 000 por clase) con división física 80/10/10 ya provista por la fuente (12 000/1 500/1 500). El dataset de detección combinado suma 17 250 imágenes: 13 800 train / 1 725 val / 1 725 test, con división 80/10/10 determinista (semilla 42) por fuente, estratificada al venir cada fuente balanceada por clase. Las particiones son físicas (archivos distintos), por lo que no hay riesgo de fuga por grupo. La clase **normal** solo existe en la fuente sintética: es la clase mayoritaria del clasificador de dominio (gate) y no participa en detección.

Limitaciones conocidas. (i) Las cajas de las fuentes sintética y SteelDefectX se derivaron de metadata y segmentación OpenCV, no de anotación humana; solo NEU-DET aporta cajas reales — las métricas de detección son, por tanto, exploratorias y no un benchmark estándar. (ii) La fuente sintética domina el volumen: existe riesgo de *sim-to-real gap*. (iii) Todas las imágenes son en escala de grises o casi; escenas a color muy saturadas quedan fuera de dominio por diseño (el gate las rechaza). (iv) NEU-DET oficial contiene archivos faltantes o corruptos; de los 1 800 originales se recuperaron 1 770 pares imagen/anotación desde un mirror público.

3. Metodología

Arquitectura en dos etapas. (1) Un *gate de dominio* (MobileNetV3-Small [4], 1.5M parámetros) clasifica la escena en 5 clases y decide si la imagen corresponde a una superficie metálica en dominio; (2) si lo es, un *detector YOLOv8n* [7] (3.0M parámetros) localiza y clasifica cada defecto. Se eligió MobileNetV3-Small por su relación precisión/tamaño para inferencia en navegador, y YOLOv8n por ser la variante liviana de Ultralytics, exportable a ONNX y con velocidad suficiente para tiempo real. Ambos parten de pesos preentrenados (ImageNet y COCO, respectivamente): transfer learning, dado el tamaño moderado del dataset.

Preprocesamiento. Imágenes redimensionadas a 256×256 ; el canal de grises se replica a RGB para aprovechar los pesos ImageNet; normalización con media/desviación de ImageNet en clasificación y escalado a $[0, 1]$ en YOLO. La web aplica exactamente el mismo preprocesamiento en JavaScript.

Hiperparámetros (Tabla 2): elegidos por recomendación de los autores de cada arquitectura y ajuste empírico acotado — no se hizo barrido sistemático por costo de GPU. En todos los casos semilla fija (42) y early stopping sobre la métrica de validación (F1 macro en clasificación, mAP50 en detección); el checkpoint final se selecciona por validación y el split de test se evalúa *una sola vez*.

Cuadro 2: Hiperparámetros clave de entrenamiento.

Parámetro	Clasificadores	YOLOv8n
Optimizador	AdamW	SGD (momentum 0.937)
Learning rate	3×10^{-4}	0.01 (cosine)
Weight decay	10^{-4}	5×10^{-4}
Batch size	32	64
Épocas máx. / paciencia	20 / 5	60 / 15
Data augmentation	flips, rotación, jitter	HSV, rotación 10°, traslación, escala, flips, mosaico 0.5, mixup 0.1
Precisión mixta (AMP)	Sí	Sí

Reproducibilidad. Todo el pipeline corre por línea de comandos (`src/download_datasets.py` → `prepare_combined_dataset.py` → `train.py` / `train_yolo.py` → `evaluate.py` / `export_*.py`), con las mismas semillas e hiperparámetros del notebook de entrenamiento (`entrenar_modelos.ipynb`), y existe un notebook de inferencia solo-CPU (`notebooks/inferencia_cpu.ipynb`). El entrenamiento en GPU puede variar mínimamente por operaciones no determinísticas de CUDA/AMP.

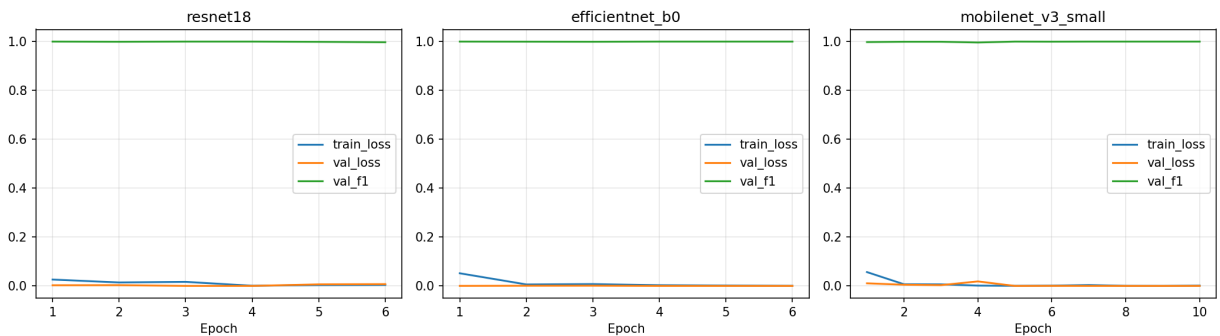
4. Resultados

4.1. Clasificación (tarea principal)

La Tabla 3 resume la evaluación final sobre el split de test (1 500 imágenes, no usado en entrenamiento ni en selección de checkpoint). Los tres modelos con transfer learning alcanzan F1 macro = 1.0000; la CNN baseline entrenada desde cero (entrenada por Jennifer Suarez Gutierrez) llega a 0.9987 con solo 391K parámetros.

Cuadro 3: Resultados de clasificación sobre test (5 clases).

Modelo	Parámetros	Épocas	Accuracy	F1 macro
ResNet-18 [5]	11.2M	7	1.0000	1.0000
EfficientNet-B0 [6]	4.0M	6	1.0000	1.0000
MobileNetV3-Small	1.5M	6	1.0000	1.0000
CNN baseline (desde cero)	391K	17	0.9987	0.9987

**Figura 1:** Curvas de entrenamiento (pérdida train/val y F1 de validación) de los tres modelos preentrenados: convergencia en pocas épocas, típica del transfer learning, sin brecha train–val que sugiera sobreajuste.

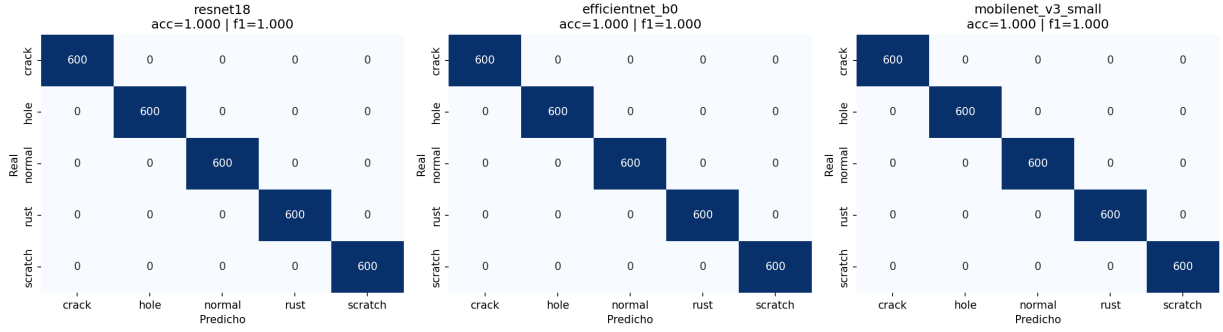


Figura 2: Matrices de confusión sobre test: diagonal pura (600/600 por clase) en los tres preentrenados.

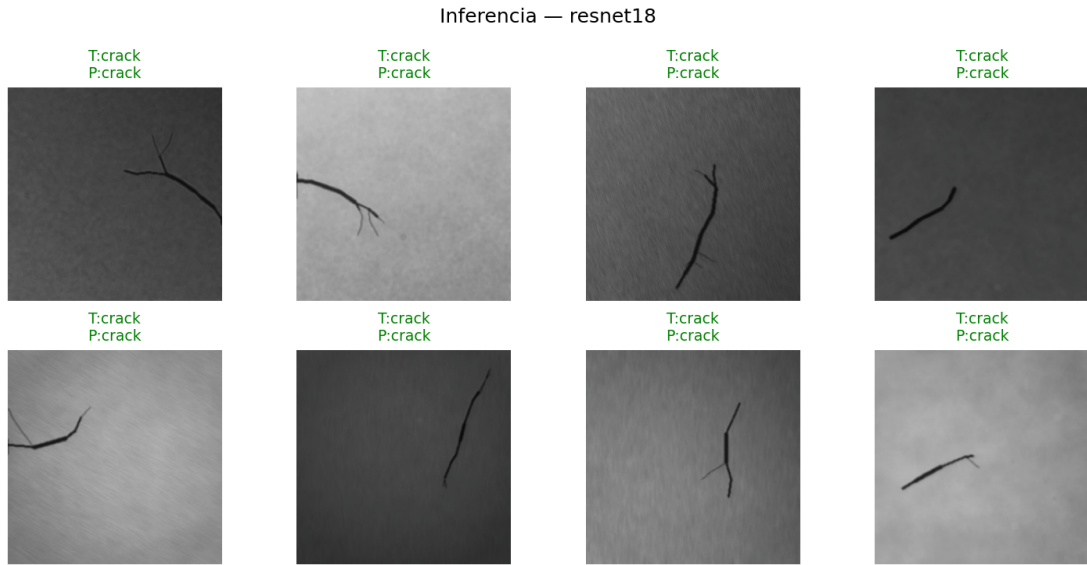


Figura 3: Ejemplos de inferencia (ResNet-18): predicciones correctas sobre imágenes de test (T = etiqueta real, P = predicción).

4.2. Detección de objetos (YOLOv8n)

Evaluación única sobre el split de test del dataset combinado de tres fuentes (1 725 imágenes, incluidas 177 reales de NEU-DET con cajas anotadas): **mAP50** = 0.739, mAP50-95 = 0.488, precisión 0.754, recall 0.663 (Tabla 4, Fig. 4).

Cuadro 4: AP por clase del detector sobre test.

Clase	AP50	AP50-95
hole (perforación)	0.846	0.645
crack (grieta)	0.791	0.541
rust (óxido)	0.677	0.407
scratch (rayón)	0.643	0.361
Global (mAP)	0.739	0.488

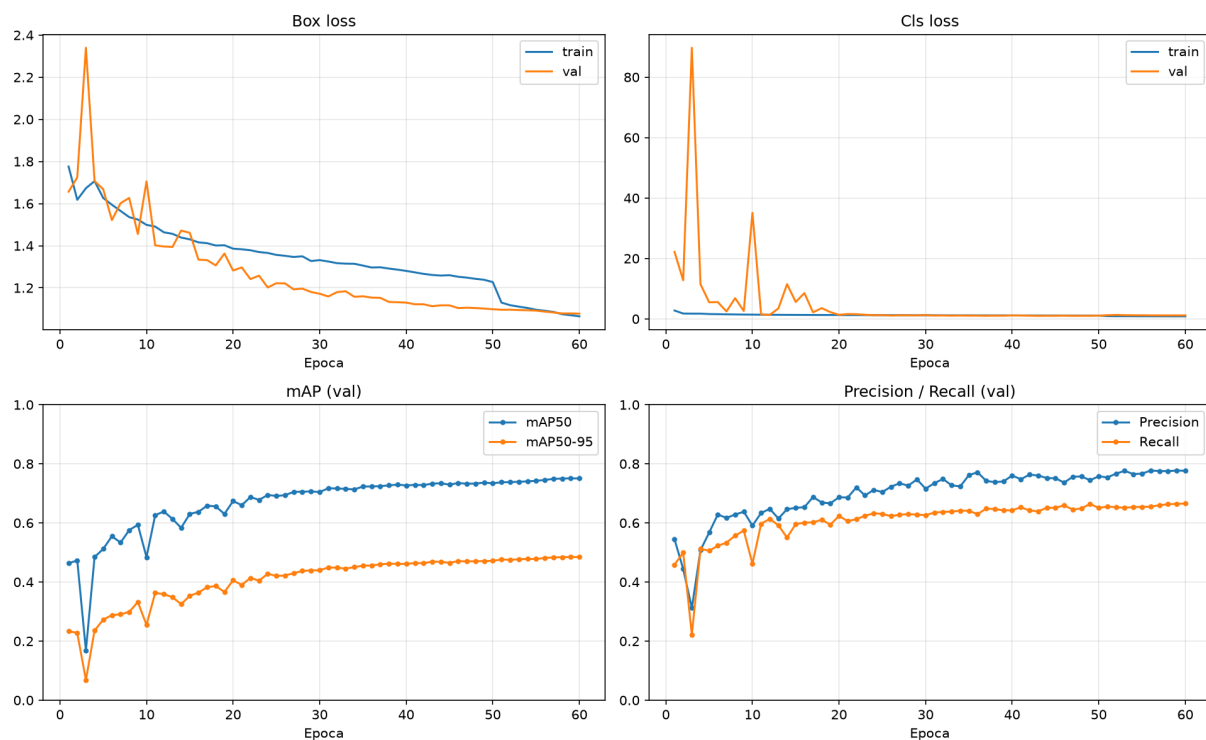


Figura 4: Curvas del detector (60 épocas, 1.15 h en RTX 5060 Ti): pérdidas box/cls en train y val, y evolución de mAP50 / mAP50-95 y precisión/recall en validación. El mAP50 aún subía al final, sin activar el early stopping.

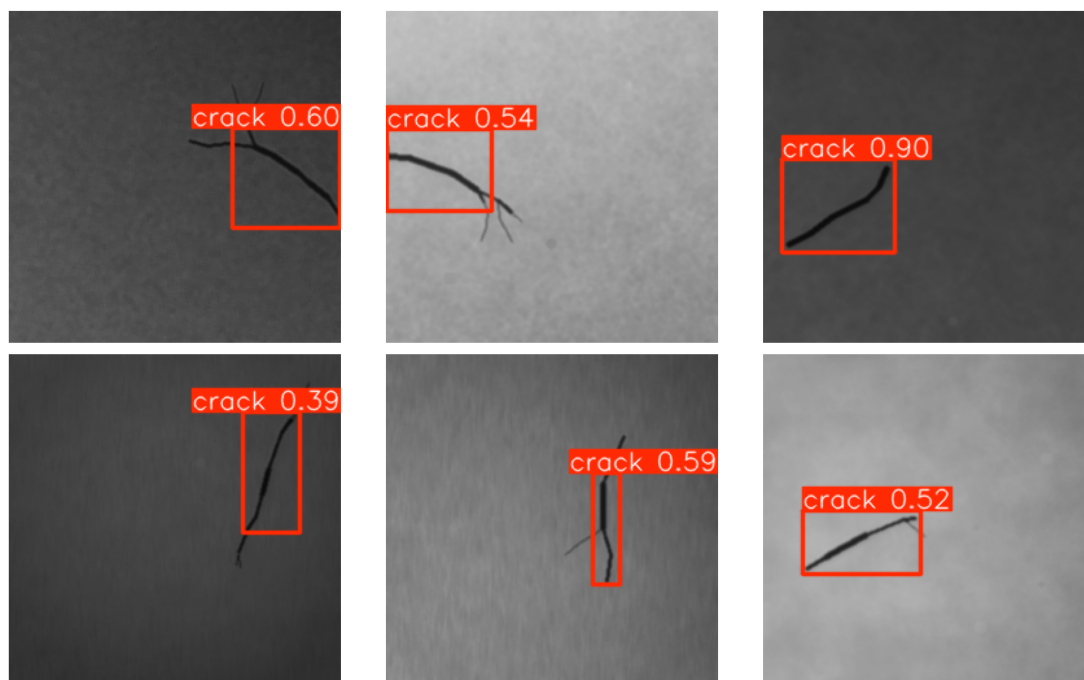


Figura 5: Detecciones del modelo sobre imágenes de test: cada defecto se acota con su caja, clase y confianza.

Errores típicos e hipótesis. Los errores de la CNN baseline (2 de 1500) corresponden a defectos de bajo contraste. En detección, la clase más difícil es **scratch**: los rayones son líneas finas y alargadas cuya caja ajustada es muy sensible a desviaciones de pocos píxeles, lo que castiga

el AP50-95; `rust` sufre por la alta variabilidad de textura del óxido y su solapamiento visual con manchas de otro origen, lo que exige un umbral específico más alto en la demo web.

4.3. Demo web (despliegue)

Ambos modelos se exportan a ONNX (opset 17) con paridad numérica verificada contra PyTorch (IoU medio 1.000 entre detecciones ONNX y PyTorch) y corren en el navegador con ONNX Runtime Web (WASM) a 20–30 FPS [8]. La Fig. 6 muestra pruebas de extremo a extremo sobre la app desplegada.

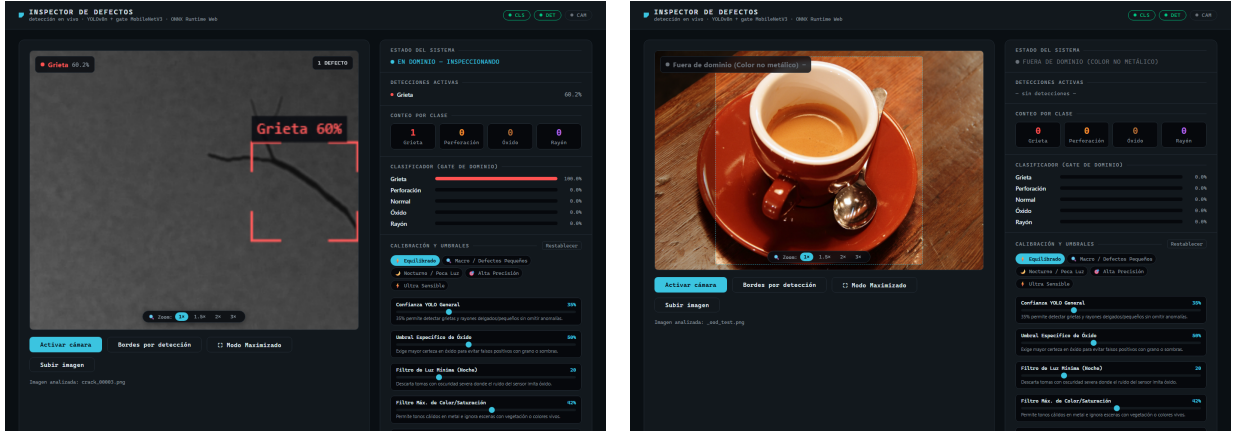


Figura 6: Testing de la demo web con el modelo desplegado. Izquierda: detección de grieta con caja y confianza. Derecha: el gate de dominio rechaza una escena no metálica (*fuera de dominio*).

5. Discusión y limitaciones

Qué funcionó mejor de lo esperado. El transfer learning saturó la tarea de clasificación sintética en 1–2 épocas con $F1 = 1.0$, y la cascada gate+detector resultó robusta en la demo con cámara real gracias a los filtros anti-ruido calibrados (luminosidad mínima, saturación máxima y umbrales por clase).

Qué no funcionó / señales a vigilar. (i) El $F1$ perfecto es una *cota superior bajo condiciones ideales*: el dataset dominante es sintético, y no afirmamos el mismo desempeño en piezas reales (*sim-to-real gap*); no hay señales de sobreajuste en las curvas, pero sí de *facilidad* del benchmark. (ii) El mAP50 del detector (0.739) no es comparable con corridas anteriores sobre el dataset de dos fuentes (0.767–0.80), porque el split de test actual incluye 177 imágenes reales de NEU-DET con cajas anotadas, un examen más exigente y más representativo. (iii) Las cajas heurísticas de dos fuentes limitan el techo de localización: parte del error AP50-95 proviene de etiquetas ruidosas, no del modelo. (iv) No detectamos fuga de datos: las particiones son físicas por fuente y el test se usó una sola vez. (v) Las curvas de mAP aún ascendían al final de las 60 épocas: el detector está sub-entrenado.

6. Trabajo futuro

1. **Reentrenar 100+ épocas con el dataset de tres fuentes:** la corrida histórica de 100 épocas (2 fuentes) mejoró 3 puntos de mAP50 frente a 60 épocas; repetirla sobre las 17 250 imágenes actuales.
2. **Anotación humana de una muestra de cajas:** revisar manualmente 500–1 000 cajas heurísticas (empezando por `scratch` y `rust`) para medir cuánto del error es ruido de etiquetas.

3. **Más datos reales y condiciones adversas:** incorporar GC10-DET y capturas propias con poca luz y cámara de planta, hoy sub-representadas, para cerrar el sim-to-real gap.
4. **Cuantización INT8 del ONNX** y WebGPU como backend de ONNX Runtime Web, para sostener 30+ FPS en hardware modesto.

7. Conclusiones

El proyecto entrega un sistema completo —datos, entrenamiento reproducible, evaluación honesta y despliegue web en tiempo real— para detección y clasificación de defectos en superficies metálicas. Los clasificadores resuelven la tarea sintética con $F1 = 1.0$ y el detector localiza defectos con $mAP50 = 0.739$ sobre un test que ya incluye imágenes reales anotadas. El principal aprendizaje es que el cuello de botella ya no es el modelo sino la calidad de las etiquetas de localización y la brecha sim-to-real, lo que marca el camino del trabajo futuro.

Referencias

- [1] Song, K., & Yan, Y. (2013). *A noise robust method based on completed local binary patterns for hot-rolled steel strip surface defect detection*. Applied Surface Science, 285, 858–864. <https://doi.org/10.1016/j.apsusc.2013.09.002>
- [2] Zhao, X. et al. (2024). *SteelDefectX: A Real-World Benchmark Dataset for Steel Surface Defect Detection*. Hugging Face Datasets. <https://huggingface.co/datasets/Zhaosxian/SteelDefectX>
- [3] Abbas, T. *Synthetic Industrial Metal Surface Defects* (15 000 imágenes, CC BY 4.0). Kaggle Datasets. <https://www.kaggle.com/datasets/tatheerabbas/synthetic-industrial-metal-surface-defects>
- [4] Howard, A. et al. (2019). *Searching for MobileNetV3*. ICCV 2019.
- [5] He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. CVPR 2016.
- [6] Tan, M., & Le, Q. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. ICML 2019.
- [7] Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8*. <https://github.com/ultralytics/ultralytics>
- [8] ONNX Runtime / ONNX Runtime Web. Microsoft. <https://onnxruntime.ai/>
- [9] Gebru, T. et al. (2018). *Datasheets for Datasets*. arXiv:1803.09010.